

LAYERED REPRESENTATIONS FROM A SINGLE IMAGE

Full Technical Project Documentation

Submitted by : Anish Maity

Roll: 21f3000417

1. Executive Summary

This project implements an end-to-end computer-vision pipeline that converts a single RGB bitmap image into a structured layered representation. The representation contains semantic regions, relative depth ordering, mutually exclusive masks, RGBA image layers, machine-readable metadata, and optional appearance and parallax outputs.

The submitted notebook uses three pretrained models: Grounding DINO for text-conditioned detection, SAM 2.1 Hiera-Tiny for pixel-level segmentation, and Depth Anything V2 Small for monocular relative depth. The pipeline does not train these models; it composes their outputs into a single representation.

In the executed example, the notebook reports 8 raw detections, 6 detections after duplicate removal, 6 valid masks after mask filtering, 4 non-overlapping foreground layers after depth-aware overlap resolution, and 5 final layers after adding the complementary background.

2. Problem Statement and Motivation

Given one bitmap RGB image, generate a stack of interpretable and re-composable layers. The intended layer representation should support semantic grouping, near-to-far depth order, and—optionally—an intrinsic appearance split such as albedo and shading.

2.1 Deliverable Mapping

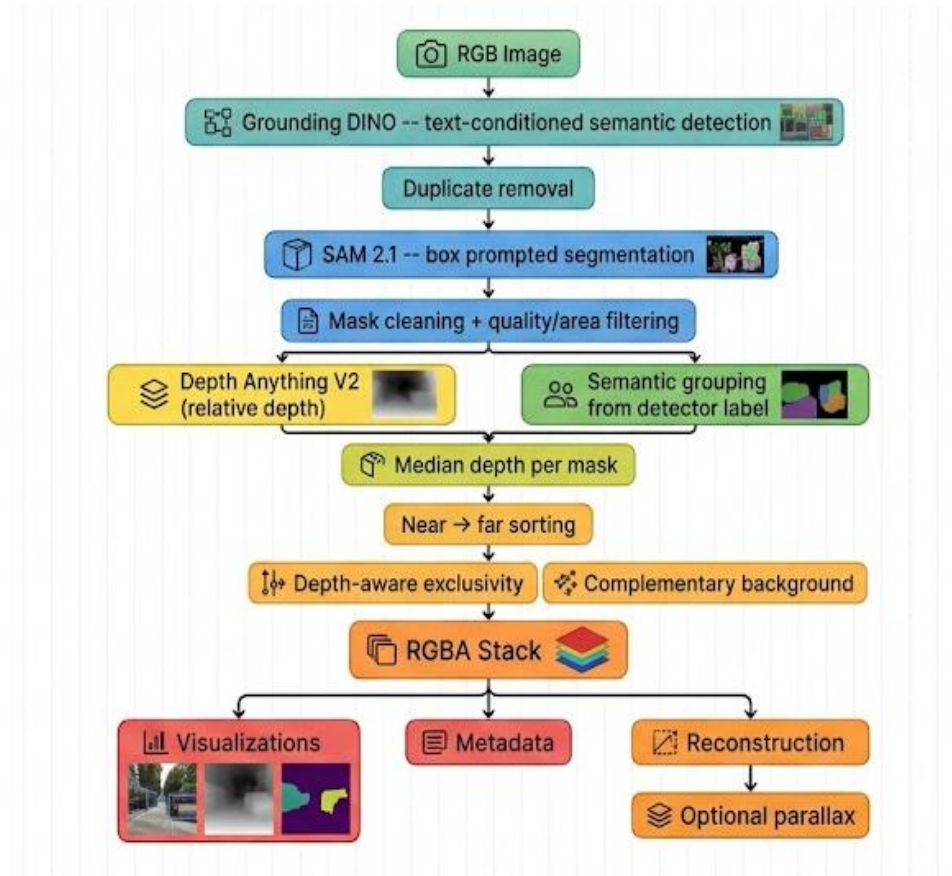
Requirement	Implementation in notebook	Status
Semantic grouping	Grounding DINO labels mapped to foreground_object, furniture, and background_stuff.	Implemented
Depth order	Depth Anything V2 relative depth; median depth inside each mask; sorting.	Implemented
RGBA layers	Original RGB pixels retained inside binary alpha masks.	Implemented
Background	Complement of the union of foreground masks.	Implemented
Albedo + shading	Gaussian-blur-based approximate intrinsic split.	Optional stretch
Re-composition	Alpha compositing of background plus far-to-near foreground.	Implemented
Animation/parallax	Depth-dependent horizontal shifts of transparent layers.	Optional demo

2.2 Applications

- Layer-wise image editing
- 2.5D parallax and pseudo-3D animation
- Scene decomposition for downstream vision tasks
- Object/background manipulation

- Approximate appearance decomposition

3. System Architecture



The architecture is intentionally modular. Detection proposes semantic regions, segmentation determines their pixels, depth supplies ordering, and post-processing converts the independent model outputs into a consistent layer stack.

4. Environment, Dependencies and Configuration

The notebook was executed in Google Colab with CUDA. Its runtime output reports PyTorch 2.11.0+cu128 and an NVIDIA Tesla T4.

Component	Notebook configuration
Packages	opencv-python, pillow, numpy, matplotlib, scipy, scikit-learn, transformers, supervision, SAM 2
Device	CUDA / Tesla T4 when available
Random seed	42
Box threshold	0.35
Text threshold	0.25

Duplicate IoU threshold	0.70
Duplicate containment threshold	0.85
Minimum area ratio	0.001
Maximum area ratio	0.80
Minimum mask score	0.20
Minimum connected-component size	100 pixels

4.1 Pretrained Models

Pipeline stage	Model ID	Role
Detection	IDEA-Research/grounding-dino-base	Open-vocabulary text-conditioned detection
Segmentation	facebook/sam2.1-hiera-tiny	Prompted image segmentation
Depth	depth-anything/Depth-Anything-V2-Small-hf	Monocular relative depth

5. Input Image and Semantic Vocabulary

The executed image has width 547 pixels and height 365 pixels. The notebook uses a compact text vocabulary containing people, vehicles, animals, furniture, and scene/background categories.

```

person . car . bicycle . motorcycle . bus . truck .
dog . cat . bird . chair . table . sofa .
tree . building . road . sky . grass . mountain .
water . sand . wall .

```

Semantic group	Labels
foreground_object	person, car, bicycle, motorcycle, bus, truck, dog, cat, bird
furniture	chair, table, sofa
background_stuff	tree, building, road, sky, grass, mountain, water, sand, wall

6. Stage 1 — Grounding DINO Detection

Grounding DINO receives the RGB image and the text prompt. The notebook converts its output into records containing a normalized label, confidence score, and bounding box. This provides semantic proposals without training a task-specific detector.

#	Label	Score	Bounding box
1	dog	0.692	[21.7, 70.9, 230.4, 365.0]
2	cat	0.367	[214.8, 143.7, 389.1, 364.9]
3	sky	0.432	[0.3, 0.1, 546.4, 278.5]
4	person	0.379	[205.7, 86.6, 373.8, 363.2]
5	grass	0.369	[0.3, 214.7, 547.3, ...]

			361.5]
6	person	0.413	[199.0, 87.2, 444.0, 364.0]
7	grass	0.394	[0.1, 261.9, 547.0, 364.8]
8	cat	0.364	[219.4, 143.7, 432.6, 365.6]

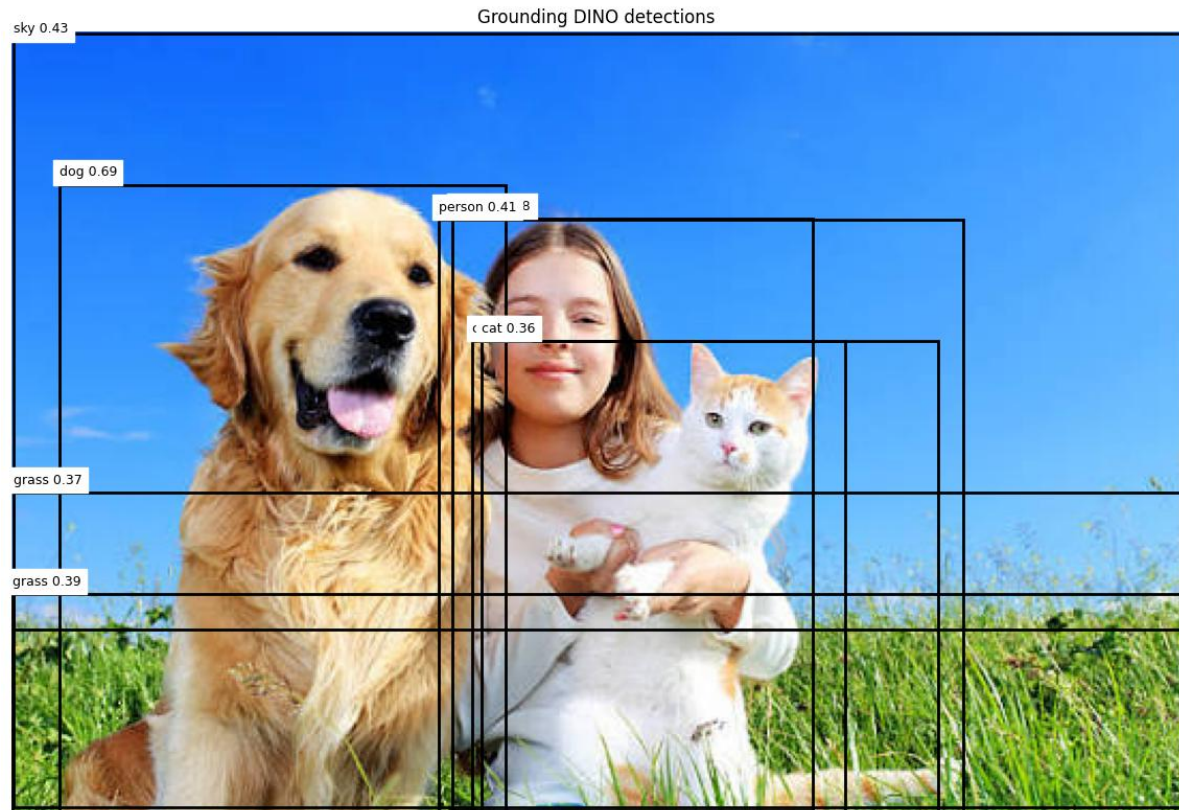


Figure 1. Grounding DINO detections.

7. Stage 2 — Duplicate Detection Removal

Same-label detections are compared after confidence sorting. A candidate is discarded when it has sufficient overlap or containment with an already retained detection.

$\text{IoU} = \text{intersection area} / \text{union area}$

$\text{containment} = \text{intersection area} / \text{area of smaller box}$

duplicate if:

same label AND ($\text{IoU} \geq 0.70$ OR $\text{containment} \geq 0.85$)

Executed result: 8 raw detections → 6 retained detections.

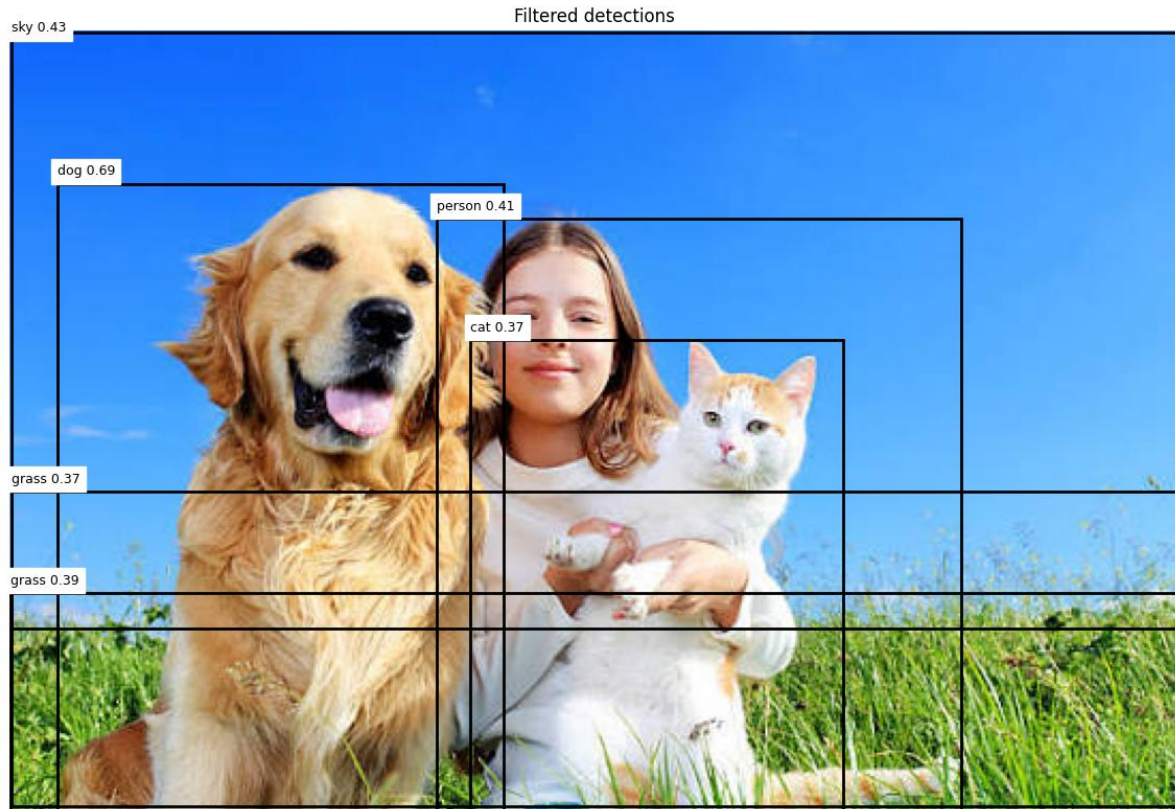


Figure 2. Detections after duplicate removal.

8. Stage 3 — SAM 2.1 Segmentation

Each retained bounding box is supplied to SAM 2.1. Multiple candidate masks are produced and the highest-scoring mask is selected.

```

predictor.set_image(image_np)

for each detection:
    masks, scores, _ = predictor.predict(
        box=detection_box,
        multimask_output=True
    )
    best_mask = masks[argmax(scores)]

```

Executed result: 6 segmented layers.

9. Stage 4 — Mask Cleaning and Filtering

Segmentation masks are refined with a 3×3 morphological opening and closing. Connected components smaller than 100 pixels are removed. The remaining masks are filtered using area ratio and SAM confidence.

Layer	Label	Area ratio	Mask score
01	dog	19.162%	0.976
02	sky	47.925%	0.971
03	person	17.390%	0.879
04	grass	10.799%	0.949
05	grass	11.295%	0.915
06	cat	9.875%	0.848

All six masks pass the notebook's configured filtering criteria in the executed example.

10. Stage 5 — Monocular Relative Depth

Depth Anything V2 Small estimates a full-image depth map. The notebook explicitly uses this as relative depth, not metric distance. The predicted values are normalized to $[0,1]$ for visualization and comparison.

Reported depth-map shape: (365, 547); normalized range: 0.0 to 1.0. The notebook documents larger values as farther regions.



Figure 3. Depth Anything V2 relative depth map.

11. Stage 6 — Depth Assignment and Ordering

Depth is sampled only inside each SAM mask. The median depth is used because it is more robust to outliers than an individual pixel.

```
values = depth_map[layer["mask"]]
layer["depth"] = median(values)
layers.sort(key=lambda x: x["depth"])    # near -> far
```

Initial order	Label	Median depth	Group
1	sky	0.1278	background_stuff
2	grass	0.3943	background_stuff
3	grass	0.4011	background_stuff
4	person	0.5957	foreground_object
5	cat	0.6210	foreground_object
6	dog	0.8351	foreground_object

These values reproduce the notebook's own relative-depth convention. They should not be interpreted as physical distances.

12. Stage 7 — Depth-Aware Exclusive Masks

SAM masks can overlap. For a clean layered representation, pixels already claimed by an earlier layer are removed from later layers. The notebook processes layers in its computed near-to-far order.

```
used = all_false_mask

for layer in layers:
    exclusive_mask = layer["mask"] & (~used)
    used |= exclusive_mask
```

Executed result: 6 candidate masks → 4 non-empty, mutually exclusive foreground layers.

13. Stage 8 — Background Completion

The background mask is defined as the complement of the foreground union. This gives full pixel coverage and makes subsequent reconstruction straightforward.

```
background_mask = ~foreground_union
```

The background is assigned depth 1.0 and appended as the final layer in the stored output stack.

14. Stage 9 — RGBA Layer Construction

Every final mask is converted to an RGBA image. RGB values are copied from the original image where the mask is true, while alpha is 255 for owned pixels and 0 for transparent pixels.

```
rgba = zeros(H, W, 4)
rgba[:, :, :3] = image_rgb * mask
```

```
rgba[:, :, 3] = mask * 255
```

This preserves the original appearance of each owned pixel while making each semantic region independently composable.

15. Generated Outputs and Folder Structure

```
/content/results/  
├── layers/  
│   ├── layer_01_sky.png  
│   ├── layer_02_grass.png  
│   ├── layer_03_person.png  
│   ├── layer_04_dog.png  
│   └── layer_05_background.png  
├── visualizations/  
│   ├── layer_masks.png  
│   ├── scene_layer_map.png  
│   ├── reconstruction.png  
│   ├── reconstructed.png  
│   └── parallax_demo.png  
├── depth/  
│   ├── depth_map.npy  
│   └── depth_map.png  
├── intrinsic/  
│   ├── <layer>_albedo.png  
│   └── <layer>_shading.png  
└── metadata.json
```

The printed filenames above are the final RGBA outputs reported by the executed notebook. The intrinsic directory contains albedo and shading files for every final layer.

16. Visualization and Scene Map

The notebook generates a combined layer-mask visualization and a depth-aware scene map. The scene map stores the layer order integer at each pixel.

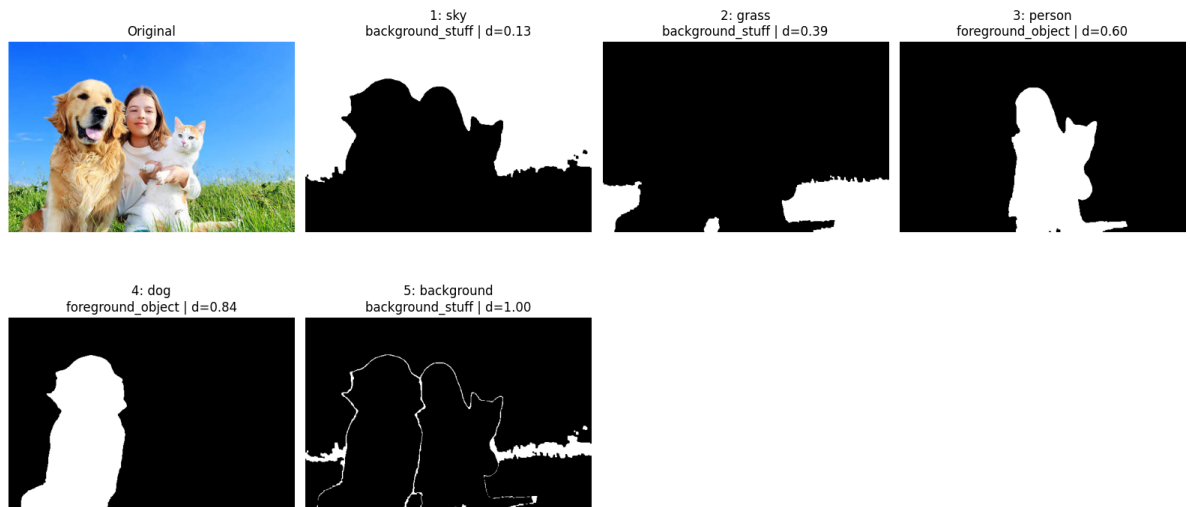


Figure 4. Layer masks, semantic groups and relative depths.

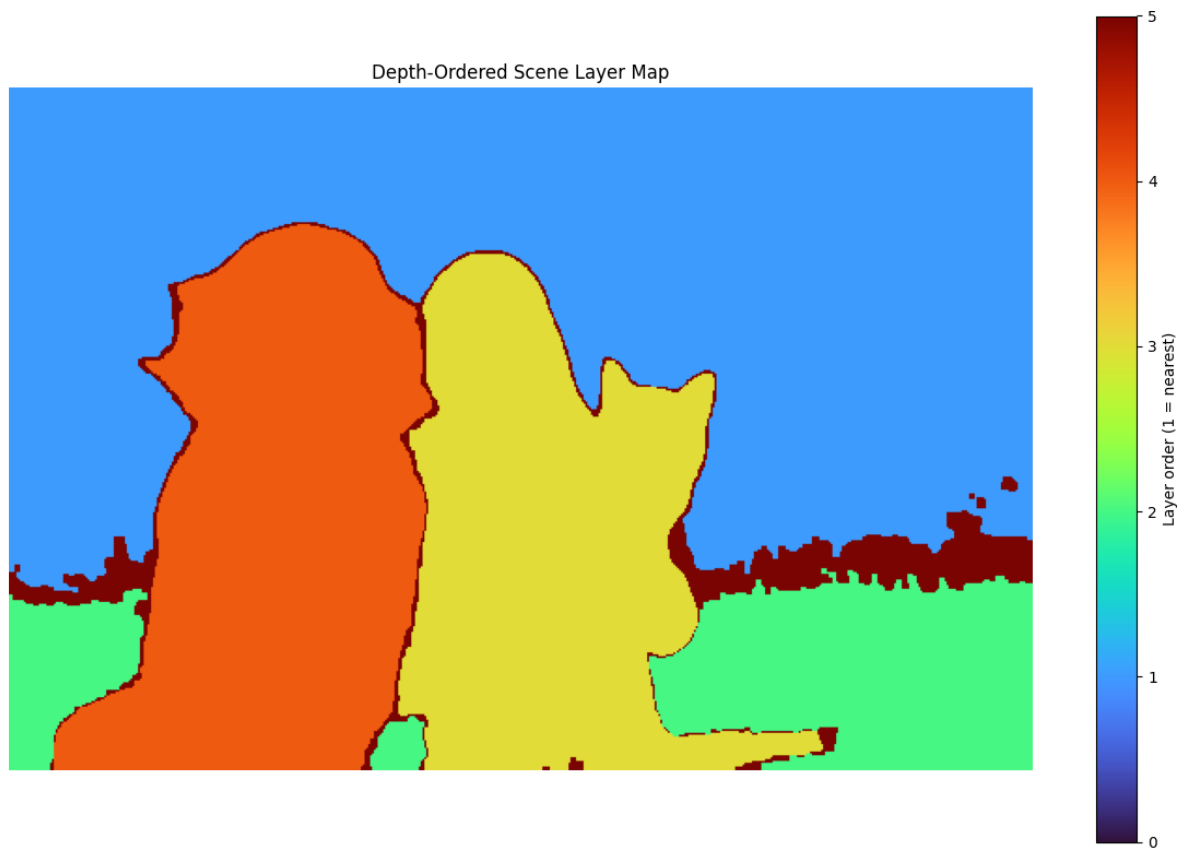


Figure 5. Depth-ordered scene layer map.

17. Re-composition and Reconstruction

Reconstruction starts from the background and composites foreground layers from far to near using alpha compositing.

```
background -> base
foreground = sorted(foreground, depth, reverse=True)

for layer in foreground:
    base = alpha_composite_rgba(base, layer["rgba"])
```



Figure 6. Original versus reconstructed image.

Metric	Reported result
MAE	0.000000
MSE	0.000000
PSNR	∞ dB

The perfect reconstruction values are expected from the construction: mutually exclusive foreground masks plus the complementary background mask partition the image, and each layer retains the original RGB values. Therefore these metrics validate lossless recomposition of the stored pixels, but do not independently validate semantic segmentation accuracy.

18. Optional Stretch — Approximate Albedo and Shading

The notebook includes a lightweight intrinsic-appearance approximation. It is explicitly described in the notebook as a stretch component, not a physically accurate intrinsic-image model.

```
luminance = 0.2126*R + 0.7152*G + 0.0722*B
illumination = GaussianBlur(luminance, sigma=15)

scale = median(illumination inside layer mask)
shading = clip(illumination / scale, 0.25, 2.0)
albedo = clip(rgb / shading, 0, 1)
```

Both albedo and shading are converted to RGBA using the same mask and saved in the intrinsic output directory.

19. Quality Report

Field	Executed result
Total layers	5
Foreground layers	4
Background present	Yes
Semantic groups	background_stuff, foreground_object
Mean foreground detection score	0.476483
Mean foreground mask score	0.935338
Reconstruction MAE	0.0
Reconstruction MSE	0.0
Reconstruction PSNR	∞ dB

20. Metadata and Machine-Readable Representation

metadata.json stores the project name, image dimensions, model identifiers, thresholds, reconstruction metrics, and per-layer information. Each layer record contains:

- id and depth_order
- label and semantic_group
- detection_score and mask_score
- relative depth
- mask area and area ratio
- bounding-box coordinates
- RGBA output path

This makes the layer stack usable by downstream programs without needing to parse the PNG filenames.

21. Final Layer Stack — Executed Example

Order	Label	Semantic group	Depth	Area
01	sky	background_stuff	0.128	47.93%
02	grass	background_stuff	0.394	11.29%
03	person	foreground_object	0.596	17.39%
04	dog	foreground_object	0.835	19.16%
05	background	background_stuff	1.000	4.23%

22. Optional 2.5D Parallax Demo

The notebook demonstrates a pseudo-3D effect by shifting each transparent layer horizontally. The shift is depth-dependent, so layers receive different amounts of motion.

```
amount = camera_shift * (1.0 - layer["depth"])
```

The executed demo uses camera_shift=25.

Pseudo-3D Parallax from Depth-Ordered Layers



Figure 7. Pseudo-3D parallax from the layered representation.

23. End-to-End Algorithm

1. Initialize the Colab environment and select CUDA when available.
2. Create output directories and load the pretrained models.
3. Load the RGB input image.
4. Run Grounding DINO using the compact semantic vocabulary.
5. Remove obvious same-label duplicate detections using IoU/containment rules.
6. Run SAM 2.1 for each retained box and select the highest-scoring mask.
7. Clean masks and apply area/score filtering.
8. Estimate the full-image relative depth map.
9. Compute median depth inside each mask.
10. Sort layers according to the notebook's near-to-far depth convention.
11. Remove overlapping pixels from later layers to make masks exclusive.
12. Create the complementary background mask.
13. Convert every layer to RGBA and save PNG files.
14. Generate layer visualizations and a scene map.
15. Recompose the image from background plus far-to-near foreground layers.
16. Compute MAE, MSE and PSNR.
17. Optionally generate approximate albedo/shading layers.

18. Save metadata and optionally generate the parallax demo.

24. Strengths of the Current Implementation

- Combines detection, segmentation and depth into one coherent representation.
- Uses text-conditioned detection, allowing the semantic vocabulary to be changed without retraining.
- Uses SAM 2.1 to obtain pixel-level masks rather than relying on bounding boxes.
- Measures depth inside segmentation masks rather than entire boxes.
- Explicitly resolves mask overlaps.
- Guarantees complete pixel coverage with a complementary background.
- Exports standard RGBA PNG layers and machine-readable metadata.
- Includes reconstruction and a practical parallax demonstration.
- Clearly labels the intrinsic decomposition as approximate.

25. Limitations and Caveats

- The submitted notebook demonstrates the method on a single executed image; there is no formal multi-image benchmark.
- No ground-truth semantic masks or metric depth annotations are used, so segmentation/depth accuracy is not quantitatively established.
- The compact vocabulary limits which categories can be discovered.
- Detection errors propagate to segmentation and therefore to the final layer stack.
- Relative depth is not metric distance.
- The perfect reconstruction score is a partition/recomposition consistency check, not a segmentation-quality score.
- The albedo/shading stage is heuristic and not physically accurate intrinsic decomposition.
- Single-image decomposition cannot recover genuinely hidden surfaces behind foreground objects.
- The depth convention and sorting direction must remain consistent if the depth model or normalization is changed.

28. Conclusion

The submitted notebook provides a practical implementation of layered scene decomposition from a single image. Its central contribution is not a newly trained model, but a carefully connected pipeline: semantic detection proposes regions, SAM 2.1 segments them, monocular depth orders them, post-processing makes them mutually exclusive, and RGBA export turns the result into a reusable layer stack.

The implementation satisfies the core project requirements for semantic grouping, depth ordering, and RGBA layer generation, and it also provides optional albedo/shading and 2.5D parallax functionality. For a stronger research or portfolio submission, the next major

improvement should be quantitative evaluation across a defined dataset, followed by stronger occlusion handling and a dedicated intrinsic-appearance model.

Notebook Link : <https://github.com/anish-maity/Layered Representations From Single Img/blob/main/Layered Representations From Single Image.ipynb>